

Anteproyecto: La Bailoteca





Autor: Antonio Manuel Aragón Pérez

INDICE

1.- Descripción del Proyecto	2
2. Objetivos.	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3.- Estructura del Proyecto	2
4.- Tecnologías Utilizadas.	3
5.- Base de Datos.	3
6.- Casos de Uso, Descripción, Roles	4
7.- Despliegue y Control de Versiones	5
8. Justificación y Alcance	5
8.1. Justificación	6
8.2. Alcance del Proyecto	6
9. Metodología de Trabajo	7
9.1. Enfoque Ágil	7
9.2. Gestión de Riesgos	7
10. Plan de Pruebas y Calidad.	7
10.1. Tipos de Pruebas	7
10.2. Herramientas y Entorno de Pruebas	8
11.- Futuras Mejoras.	8
12.- Integrantes Grupo.	8
Antonio Manuel Aragón Pérez	8

1.- Descripción del Proyecto.

La Bailoteca es una aplicación diseñada para gestionar una academia de baile. La app facilitará todas las funciones necesarias para la administración de la academia, incluyendo el registro y autenticación de usuarios, la gestión de cursos, la administración de profesores, la inscripción de alumnos y el seguimiento de su progreso. La aplicación ofrecerá funcionalidades CRUD (Crear, Leer, Actualizar, Borrar) en todas sus secciones, proporcionando dos interfaces de usuario: una web y otra móvil, ambas conectadas a un backend.

2. Objetivos.

2.1. Objetivo General

Diseñar e implementar una plataforma integral para la gestión de una academia de baile, que abarque tanto el uso por parte de administradores y profesores, como la experiencia de los alumnos e invitados.

2.2. Objetivos Específicos

1. **Centralizar la información** de la academia (usuarios, clases, horarios, inscripciones, eventos, etc.) en un solo sistema con acceso web y móvil.
2. **Facilitar la comunicación** entre profesores y alumnos a través de notificaciones, chat interno y eventos.
3. **Automatizar la inscripción** de nuevos alumnos, permitiendo gestionar de forma sencilla las altas y bajas en las clases.
4. **Integrar un sistema de autenticación seguro** mediante Firebase, garantizando la protección de los datos y la escalabilidad del servicio.
5. **Ofrecer un panel de administración** para la gestión de profesores, clases y contenido adicional (vídeos, documentos, etc.).
6. **Permitir el seguimiento y control** de la participación y pagos (posible mejora futura) de los alumnos, así como la asistencia a eventos y clases.
7. **Establecer una base sólida** para futuras mejoras, como integración de sistemas de pago online o módulos de analíticas.

3.- Estructura del Proyecto.

• Backend:

Desarrollado en Java Spring Boot, este módulo se encargará de la lógica del negocio, la comunicación con la base de datos, la seguridad (integrando autenticación mediante Firebase) y la exposición de una API RESTful para el uso por los frontends.

- **Frontend Web:**

Implementado en React con Vite, proporcionará la interfaz de usuario para la administración y gestión de la academia.

- **Aplicación Móvil:**

Desarrollada en Android utilizando Kotlin, para dispositivos móviles, permitiendo a los usuarios interactuar con la plataforma a través de un app nativa.

- **Base de Datos:**

Almacén de datos que se utilizará para guardar la información de usuarios, cursos, profesores, horarios, etc.

- **Sistema de Autenticación:**

Se integrará Firebase para gestionar la autenticación de usuarios de forma segura y escalable.

4.- Tecnologías Utilizadas.

- **Backend:** Java Spring Boot

- **Frontend Web:** React con Vite

- **Aplicación Móvil:** Android (Kotlin)

- **Autenticación:** Firebase

- **Comunicación:** API RESTful

- **Otras:** Se utilizará Docker y docker-compose para la dockerización y despliegue del entorno de la aplicación.

5.- Base de Datos.

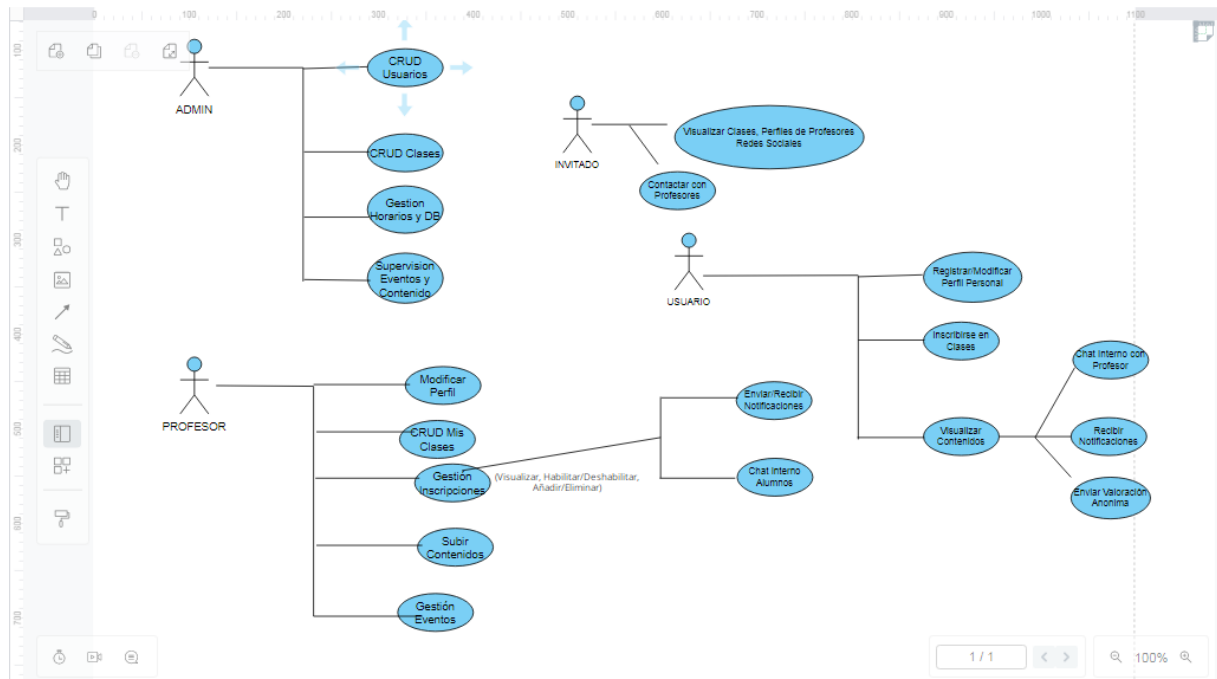
- **Tipo:** Relacional.

- **SGBD Seleccionado:** PostgreSQL.

- **Ventajas para el Proyecto:**

- **Integridad de datos:** Permite definir relaciones (por ejemplo, entre usuarios, clases y profesores) que garantizan la consistencia en la gestión de información.
- **Compatibilidad y herramientas:** Amplia compatibilidad con frameworks de Java (como Spring Boot) y disponibilidad de herramientas para administración y monitoreo.

6.- Casos de Uso, Descripción, Roles.



He intentado usar la tecnología de creación de diagramas de casos de uso de Mermaid para generarlo. En el caso de Visual Studio la versión de la que dispongo actualmente no permite el código para generar la imagen. Aun así este sería el código para verlo en después en la preview con la versión adecuada:

```
%%{init: {'theme': 'base'}}%%
---
title: Diagrama de Casos de Uso - La Bailoteca
---
graph TD
    Admin["Administrador"] -->|Usa| UC1["Registrarse"]
    Admin -->|Usa| UC2["Iniciar Sesión"]
    Admin -->|Gestiona| UC4["Gestión de Usuarios"]
    Admin -->|Administra| UC5["Gestión de Clases"]
    Admin -->|Consulta| UC14["Ver Estadísticas/Analíticas"]
    Admin -->|Acción| UC8["Cancelar Inscripción"]

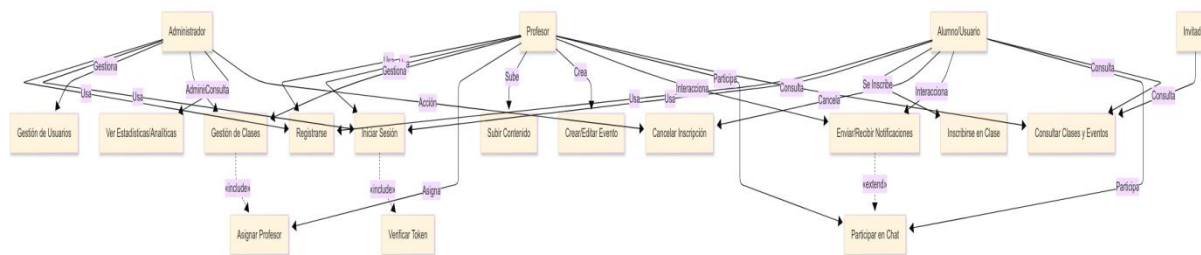
    Prof["Profesor"] -->|Usa| UC1
    Prof -->|Usa| UC2
    Prof -->|Gestiona| UC5
    Prof -->|Asigna| UC6["Asignar Profesor"]
    Prof -->|Sube| UC9["Subir Contenido"]
    Prof -->|Crea| UC10["Crear/Editar Evento"]
    Prof -->|Interacciona| UC11["Enviar/Recibir Notificaciones"]
    Prof -->|Participa| UC12["Participar en Chat"]
    Prof -->|Consulta| UC13["Consultar Clases y Eventos"]
```

```
Alumno["Alumno/Usuario"] -->|Usa| UC1
Alumno -->|Usa| UC2
Alumno -->|Se Inscribe| UC7["Inscribirse en Clase"]
Alumno -->|Cancela| UC8
Alumno -->|Interacciona| UC11
Alumno -->|Participa| UC12
Alumno -->|Consulta| UC13

Invitado["Invitado"] -->|Consulta| UC13

UC2 -->|«include»| UC3["Verificar Token"]
UC5 -->|«include»| UC6
UC11 -->|«extend»| UC12
```

Aun así en la página oficial de Mermaid con un nuevo código se ha podido generar. Pero esta página no soporta los diagramas de casos de uso, pero lo he podido adaptar en forma de Grapho y aunque no sea muy visual sería tal que así:



7.- Despliegue y Control de Versiones.

▪ Despliegue:

La aplicación se dockerizará utilizando un archivo `docker-compose.yml` y `.env`, lo que permitirá levantar todos los servicios necesarios (backend, base de datos, etc.) de forma sencilla.

▪ Control de Versiones:

Se utilizará Git para el control de versiones especificando cada commit con un mensaje claro, separando en ramas y haciendo esos commit y push a cada clase/modulo una vez testado.

8. Justificación y Alcance.

El motivo del porque elegí este proyecto se debe a que de manera personal el baile es mi hobby y en el cual muchos de los profesores de baile de esta academia los cuales saben de mi formación actual les interesaba algún tipo de app o web que permitiera centralizar y gestionar de manera más centralizada y sencilla la gestión de la academia.

8.1. Justificación

- **Demanda real:** En el sector de la enseñanza artística (academias de baile, música, etc.) existe la necesidad de digitalizar procesos de gestión que a menudo se llevan en papel o en hojas de cálculo.
- **Eficiencia y escalabilidad:** Al contar con una aplicación web y una app móvil, se simplifica la comunicación y la administración, reduciendo el uso de herramientas dispersas (WhatsApp, correos sueltos, etc.).
- **Tecnologías modernas:** El uso de Java Spring Boot para el backend, React para el frontend web y Kotlin para la aplicación móvil responde a la necesidad de tener una arquitectura robusta, mantenible y con un alto grado de escalabilidad.
- **Seguridad:** La integración con Firebase ofrece un sistema de autenticación fiable, con soporte para distintos métodos de login y protección de datos de usuario.
- **Alineación con tendencias del mercado:** Docker, docker-compose y la filosofía de microservicios o contenedores son cada vez más demandados en entornos profesionales, lo que otorga al proyecto un carácter formativo y práctico.

8.2. Alcance del Proyecto

- **Gestión de Usuarios:** Alta, baja, modificación y consulta de perfiles (roles: **ADMIN**, **PROFESOR**, **USUARIO**, **INVITADO**).
- **Administración de Clases:** Creación, modificación y eliminación de clases, con horarios y asignación de profesores.
- **Inscripciones:** Sistema que permite a los alumnos apuntarse a las clases, con control de estado y pago (aun sin implementar).
- **Eventos y Contenidos:** Creación y difusión de eventos, y subida de materiales adicionales (vídeos, PDFs, etc.).
- **Interfaces de Usuario:**
 - **Web** (React + Vite): para administración y consulta de clases y usuarios.
 - **Móvil** (Android + Kotlin): para que alumnos y profesores puedan consultar y gestionar la información en cualquier lugar.
- **Base de Datos:** PostgreSQL para almacenar de forma relacional toda la información.
- **Autenticación y Autorización:** Mediante Firebase y, opcionalmente, un sistema de roles en la base de datos para control de accesos.
- **Despliegue con Docker:** Uso de contenedores para el backend, la base de datos y otros servicios, facilitando la puesta en marcha del entorno.

9. Metodología de Trabajo.

9.1. Enfoque Ágil

- **Reuniones Periódicas (Sprints):**
 - Definir sprints de 2-3 semanas para planificar objetivos concretos (por ejemplo, primer sprint: CRUD de usuarios y autenticación; segundo sprint: gestión de clases y eventos; etc.).
 - Revisar el trabajo realizado al final de cada sprint y ajustar el backlog en función de las prioridades.
- **Tablero Kanban:**
 - Mantener un tablero virtual (Kanban) donde se visualizan las tareas pendientes, en progreso y finalizadas y así mantener una lista de tareas y facilitar la organización.
- **División de Tareas:**
 - Separadas las tareas por **módulos** de backend, frontend...
- **Comunicación Continua:**
 - Aunque sea un proyecto individual simulo con el uso de varios commits la simulación de un entorno de trabajo y la comunicación entre participantes.

9.2. Gestión de Riesgos

- **Riesgo Tecnológico:** Familiarización con tecnologías nuevas (Kotlin, Docker, etc.) y curva de aprendizaje. Por lo cual a la hora de trabajar con ellas podría haber los típicos riesgos de la falta de experiencia y trabajo con ellas
- **Riesgo de Integración:** Posibles problemas al integrar la app móvil, el frontend web y el backend con la misma API. Para intentar evitarlas se realizaran pruebas tempranas al ritmo del desarrollo.
- **Riesgo de Plazos:** Ajustar el alcance según la disponibilidad de tiempo, priorizando las funcionalidades principales.

10. Plan de Pruebas y Calidad.

10.1. Tipos de Pruebas

1. **Pruebas Unitarias (Backend):**
 - Asegurar que los métodos CRUD y la lógica de validación se comportan como se espera.
2. **Pruebas de Integración:**
 - Verificar la comunicación entre el backend (API REST) y la base de datos PostgreSQL.
 - Comprobar la correcta autenticación con Firebase (validación de tokens).
 - Asegurarse de que los endpoints devuelven la información en el formato esperado.

3. Pruebas Funcionales (Frontend y Móvil):

- Comprobar que las vistas en React y la app Android muestran y gestionan los datos correctamente.
- Validar flujos completos (por ejemplo, registro de usuario → login → inscripción en clase).

4. Pruebas de Usabilidad con Usuarios Reales:

- Reunir un pequeño grupo de usuarios (profesores, alumnos, administradores, familiares) para probar las interfaces y flujos principales.
- Recoger feedback sobre facilidad de uso, tiempos de carga, claridad de la información, etc.
- Incorporar mejoras en base a este feedback.

10.2. Herramientas y Entorno de Pruebas

- **Herramientas de Testing:** JUnit, Mockito, Jest/React Testing Library, Espresso (para Android).
- **Entorno de Pruebas:** Contenedores Docker con datos de prueba en PostgreSQL.
- **Gestión de Incidencias:** Registrar bugs y mejoras en GitHub Issues, Jira o similar.

11.- Futuras Mejoras.

- **Modulo de Gestión de Pagos:** Actualmente todo el control de los pagos se realiza en efectivo y los mismos profesores personalmente llevan el pago de sus propios alumnos. Quizás se podría implementar un sistema de pago por bizum y añadir el atributo pagado/noPagado
- **Control de Analíticas:** Desarrollar un panel de control (dashboard) para que tanto administradores como profesores puedan visualizar estadísticas relevantes, como número de inscripciones, asistencia a clases y tendencias de valoración.
- **Integración con Google Calendar u otras plataformas:** Permitir a los usuarios sincronizar sus clases y eventos con sus calendarios personales.
- **Ampliación del Chat Interno:** Incluir funcionalidades de grupo, adjuntar archivos y notificaciones en tiempo real.

12.- Integrantes Grupo.

Antonio Manuel Aragón Pérez